Технологии конструирования программного обеспечения

Отчет по лабораторной работе № 5

Группа: 241-327 Студент: Дубина Анастасия Александровна

Задание на лабораторную работу

2. Банкомат Атрибуты: ID, общая сумма денег в банкомате, вероятность отсутствия связи с банком. Операции: ввести PIN-код, внять заданную сумму, завершить работу, загрузить деньги в банкомат	Ожидание, Аутентификация пользователя, Выполнение операций, Заблокирован (денег нет)

Диаграмма классов

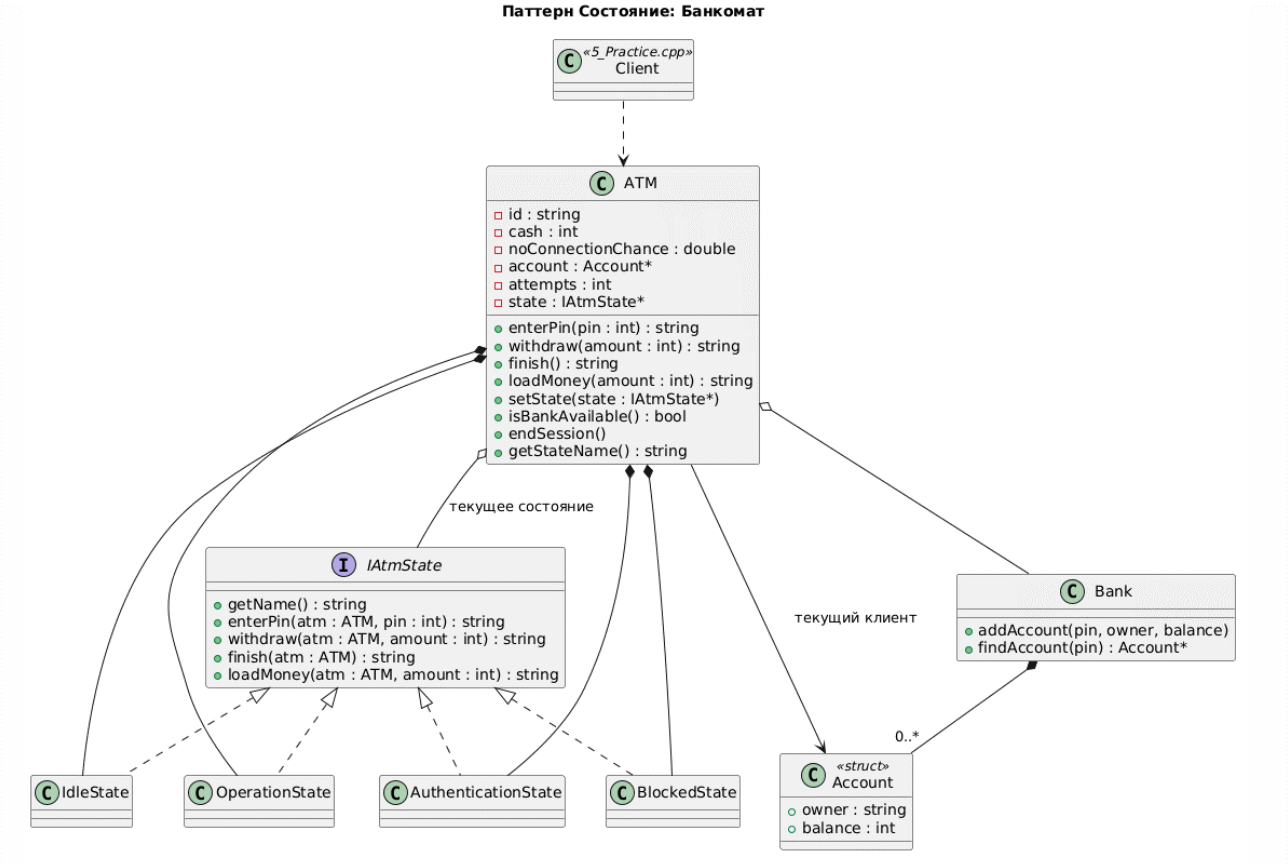


Диаграмма последовательности

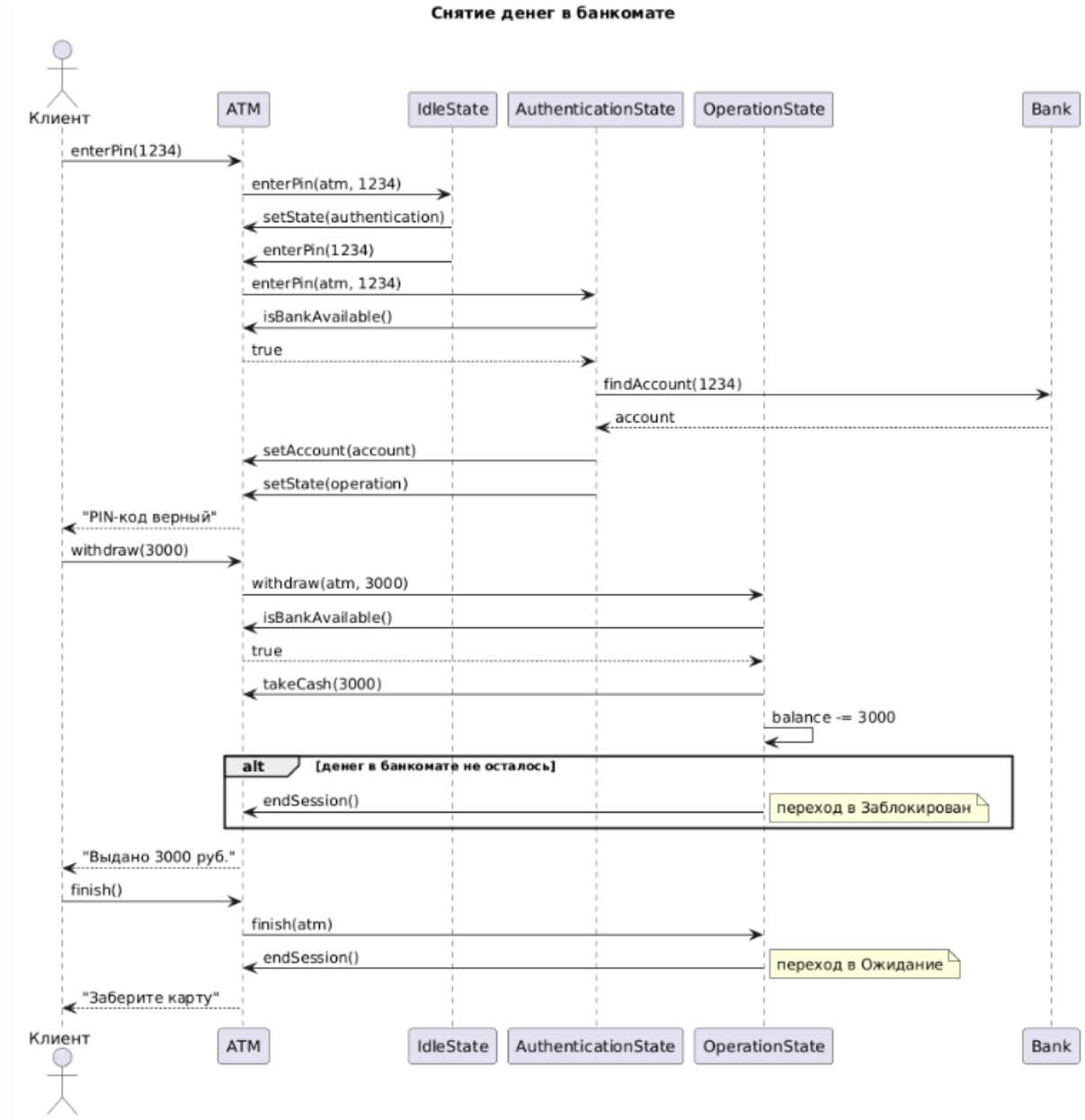
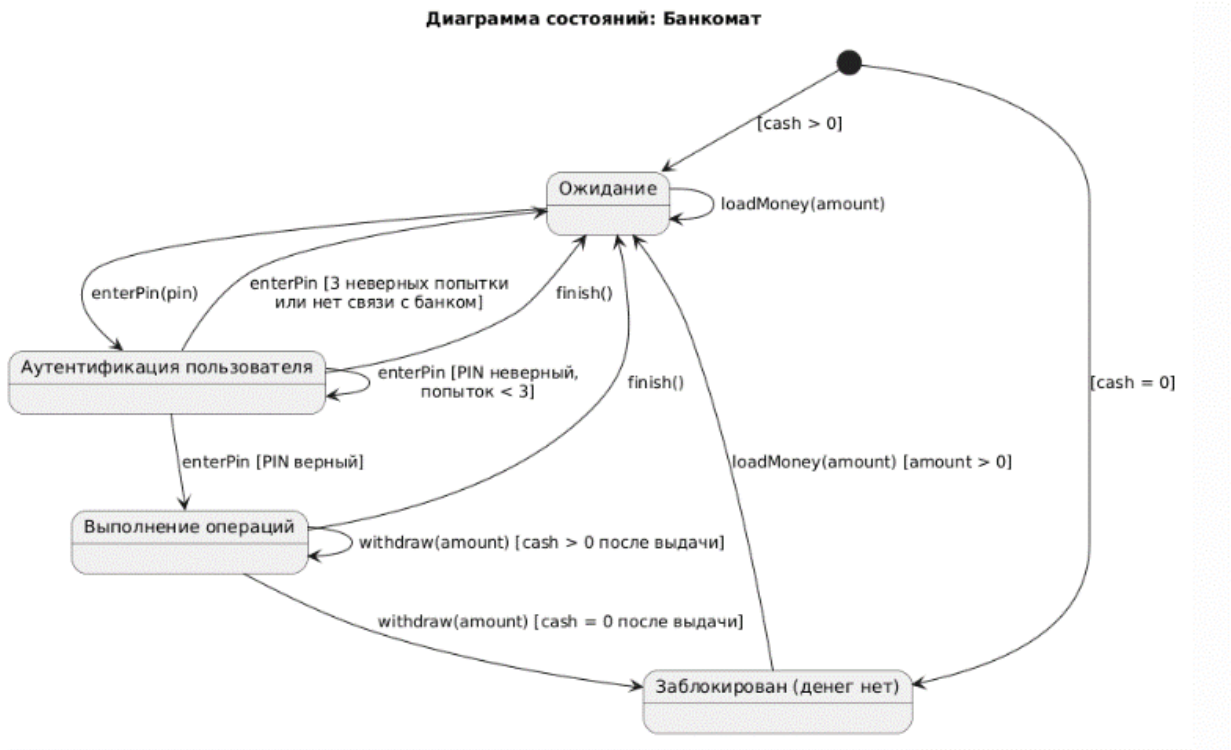


Диаграмма состояний



Исходный код программы

5_AtmState.h

```
#pragma once

#include <string>
#include <map>
#include <random>

namespace AtmSystem {

// ======================== Банк ========================

// Счет клиента банка
struct Account {
    std::string owner;
    int balance;
};

// Банк хранит счета; ключ - PIN-код карты (упрощение для лабораторной)
class Bank {
public:
    void addAccount(int pin, const std::string& owner, int balance) {
        accounts[pin] = Account{ owner, balance };
    }

    Account* findAccount(int pin) {
```

```
        auto it = accounts.find(pin);
        return it == accounts.end() ? nullptr : &it->second;
    }

private:
    std::map<int, Account> accounts;
};

class ATM;

// ======================== Интерфейс Состояние ========================

// Каждая операция банкомата возвращает сообщение для пользователя
class IAtmState {
public:
    virtual ~IAtmState() {}
    virtual std::string getName() const = 0;
    virtual std::string enterPin(ATM& atm, int pin) = 0;
    virtual std::string withdraw(ATM& atm, int amount) = 0;
    virtual std::string finish(ATM& atm) = 0;
    virtual std::string loadMoney(ATM& atm, int amount) = 0;
};

// ======================== Конкретные состояния ========================

class IdleState : public IAtmState {
public:
    std::string getName() const override { return "Ожидание"; }
    std::string enterPin(ATM& atm, int pin) override;
    std::string withdraw(ATM& atm, int amount) override;
    std::string finish(ATM& atm) override;
    std::string loadMoney(ATM& atm, int amount) override;
};

class AuthenticationState : public IAtmState {
public:
    std::string getName() const override { return "Аутентификация пользователя"; }
    std::string enterPin(ATM& atm, int pin) override;
    std::string withdraw(ATM& atm, int amount) override;
    std::string finish(ATM& atm) override;
    std::string loadMoney(ATM& atm, int amount) override;
};

class OperationState : public IAtmState {
public:
    std::string getName() const override { return "Выполнение операций"; }
    std::string enterPin(ATM& atm, int pin) override;
    std::string withdraw(ATM& atm, int amount) override;
```

```
    std::string finish(ATM& atm) override;
    std::string loadMoney(ATM& atm, int amount) override;
};

class BlockedState : public IAtmState {
public:
    std::string getName() const override { return "Заблокирован"; }
    std::string enterPin(ATM& atm, int pin) override;
    std::string withdraw(ATM& atm, int amount) override;
    std::string finish(ATM& atm) override;
    std::string loadMoney(ATM& atm, int amount) override;
};

// ======================== Конечный автомат ========================

// Банкомат хранит текущее состояние и передает ему все операции.
// Объекты состояний создаются один раз и переключаются указателем.
class ATM {
public:
    static const int MAX_ATTEMPTS = 3;

    ATM(const std::string& id, int cash, double noConnectionChance, Bank& bank, unsigned int seed)
        : id(id), cash(cash), noConnectionChance(noConnectionChance), bank(bank),
          random(seed), account(nullptr), attempts(0) {
        state = cash > 0 ? (IAtmState*)&idle : (IAtmState*)&blocked;
    }

    // Операции, которые вызывает пользователь
    std::string enterPin(int pin) { return state->enterPin(*this, pin); }
    std::string withdraw(int amount) { return state->withdraw(*this, amount); }
    std::string finish() { return state->finish(*this); }
    std::string loadMoney(int amount) { return state->loadMoney(*this, amount); }

    std::string getId() const { return id; }
    int getCash() const { return cash; }
    std::string getStateName() const { return state->getName(); }

    // Методы ниже используют классы состояний
    void setState(IAtmState* newState) { state = newState; }
    IAtmState* getIdleState() { return &idle; }
    IAtmState* getAuthenticationState() { return &authentication; }
    IAtmState* getOperationState() { return &operation; }
    IAtmState* getBlockedState() { return &blocked; }

    void addCash(int amount) { cash += amount; }
    void takeCash(int amount) { cash -= amount; }

    Bank& getBank() { return bank; }
```

```
    Account* getAccount() { return account; }
    void setAccount(Account* value) { account = value; }

    int getAttempts() const { return attempts; }
    void setAttempts(int value) { attempts = value; }

    // Есть ли связь с банком (случайное событие)
    bool isBankAvailable() {
        std::uniform_real_distribution<double> dist(0.0, 1.0);
        return dist(random) >= noConnectionChance;
    }

    // Завершение сеанса: сброс данных клиента и переход в нужное состояние
    void endSession() {
        account = nullptr;
        attempts = 0;
        state = cash > 0 ? (IAtmState*)&idle : (IAtmState*)&blocked;
    }

private:
    std::string id;
    int cash;
    double noConnectionChance;
    Bank& bank;
    std::mt19937 random;
    Account* account;
    int attempts;

    IdleState idle;
    AuthenticationState authentication;
    OperationState operation;
    BlockedState blocked;
    IAtmState* state;
};

// ======================== Реализация состояний ========================

// --- Ожидание ---

inline std::string IdleState::enterPin(ATM& atm, int pin) {
    atm.setState(atm.getAuthenticationState());
    return atm.enterPin(pin);
}

inline std::string IdleState::withdraw(ATM&, int) {
    return "Сначала введите PIN-код";
}
```

```
inline std::string IdleState::finish(ATM&) {
   return "Банкомат уже в режиме ожидания";
}

inline std::string IdleState::loadMoney(ATM& atm, int amount) {
   if (amount <= 0) return "Сумма загрузки должна быть больше нуля";
   atm.addCash(amount);
   return "Загружено " + std::to_string(amount) + " руб.";
}

// --- Аутентификация пользователя ---

inline std::string AuthenticationState::enterPin(ATM& atm, int pin) {
   if (!atm.isBankAvailable()) {
      atm.endSession();
      return "Нет связи с банком, попробуйте позже";
   }
   Account* account = atm.getBank().findAccount(pin);
   if (account != nullptr) {
      atm.setAccount(account);
      atm.setState(atm.getOperationState());
      return "PIN-код верный. Здравствуйте, " + account->owner;
   }
   atm.setAttempts(atm.getAttempts() + 1);
   if (atm.getAttempts() >= ATM::MAX_ATTEMPTS) {
      atm.endSession();
      return "Неверный PIN-код. Попытки исчерпаны, карта изъята";
   }
   return "Неверный PIN-код. Осталось попыток: " +
         std::to_string(ATM::MAX_ATTEMPTS - atm.getAttempts());
}

inline std::string AuthenticationState::withdraw(ATM&, int) {
   return "Сначала введите верный PIN-код";
}

inline std::string AuthenticationState::finish(ATM& atm) {
   atm.endSession();
   return "Операция отменена, заберите карту";
}

inline std::string AuthenticationState::loadMoney(ATM&, int) {
   return "Банкомат занят клиентом, загрузка невозможна";
}

// --- Выполнение операций ---

inline std::string OperationState::enterPin(ATM&, int) {
```

```
    return "PIN-код уже введен";
}

inline std::string OperationState::withdraw(ATM& atm, int amount) {
    if (amount <= 0 || amount % 100 != 0) return "Сумма должна быть кратна 100 руб.";
    if (amount > atm.getCash()) return "В банкомате недостаточно денег";
    if (amount > atm.getAccount()->balance) return "Недостаточно средств на счете";
    if (!atm.isBankAvailable()) return "Нет связи с банком, операция не выполнена";

    atm.getAccount()->balance -= amount;
    atm.takeCash(amount);
    std::string message = "Выдано " + std::to_string(amount) + " руб. Остаток на счете: " +
                          std::to_string(atm.getAccount()->balance) + " руб.";
    if (atm.getCash() == 0) {
        atm.endSession();
        message += " Деньги в банкомате закончились, банкомат заблокирован";
    }
    return message;
}

inline std::string OperationState::finish(ATM& atm) {
    atm.endSession();
    return "Работа завершена, заберите карту";
}

inline std::string OperationState::loadMoney(ATM&, int) {
    return "Банкомат занят клиентом, загрузка невозможна";
}

// --- Заблокирован (денег нет) ---

inline std::string BlockedState::enterPin(ATM&, int) {
    return "Банкомат заблокирован: нет денег";
}

inline std::string BlockedState::withdraw(ATM&, int) {
    return "Банкомат заблокирован: нет денег";
}

inline std::string BlockedState::finish(ATM&) {
    return "Банкомат заблокирован: нет денег";
}

inline std::string BlockedState::loadMoney(ATM& atm, int amount) {
    if (amount <= 0) return "Сумма загрузки должна быть больше нуля";
    atm.addCash(amount);
    atm.setState(atm.getIdleState());
    return "Загружено " + std::to_string(amount) + " руб. Банкомат разблокирован";
```

```
}
}
```

5_Practice.cpp

```
#include <iostream>
#include <string>
#include <random>
#include <clocale>
#include "5_AtmState.h"

using namespace std;
using namespace AtmSystem;

// Выполняет операцию и печатает ответ и новое состояние
void show(ATM& atm, const string& action, const string& answer) {
   cout << action << " -> " << answer << endl;
   cout << "   [состояние: " << atm.getStateName() << ", в банкомате "
        << atm.getCash() << " руб.]" << endl;
}

// ====================== Тестирование ======================

class Tester {
public:
   Tester() : passed(0), failed(0) {}

   void check(bool condition, const string& name) {
      if (condition) {
         passed++;
         cout << "  [OK]   " << name << endl;
      } else {
         failed++;
         cout << "  [FAIL] " << name << endl;
      }
   }

   void printResult() const {
      cout << "Пройдено: " << passed << ", провалено: " << failed << endl;
   }

private:
   int passed;
   int failed;
};

// Банк с одним счетом: PIN 1234, баланс 10000
```

```
void fillBank(Bank& bank) {
   bank.addAccount(1234, "Иванов", 10000);
}

void testInitialState(Tester& t) {
   Bank bank;
   ATM withMoney("A1", 5000, 0, bank, 1);
   ATM empty("A2", 0, 0, bank, 1);
   t.check(withMoney.getStateName() == "Ожидание", "Банкомат с деньгами начинает в Ожидании");
   t.check(empty.getStateName() == "Заблокирован", "Пустой банкомат начинает в состоянии
Заблокирован");
}

void testIdle(Tester& t) {
   Bank bank;
   fillBank(bank);
   ATM atm("A1", 5000, 0, bank, 1);
   atm.withdraw(1000);
   t.check(atm.getStateName() == "Ожидание" && atm.getCash() == 5000, "В Ожидании снять деньги
нельзя");
   atm.loadMoney(1000);
   t.check(atm.getStateName() == "Ожидание" && atm.getCash() == 6000, "В Ожидании можно загрузить
деньги");
}

void testAuthentication(Tester& t) {
   Bank bank;
   fillBank(bank);
   ATM atm("A1", 5000, 0, bank, 1);

   atm.enterPin(1111);
   t.check(atm.getStateName() == "Аутентификация пользователя" && atm.getAttempts() == 1,
         "Неверный PIN: остаемся в Аутентификации");
   atm.enterPin(1234);
   t.check(atm.getStateName() == "Выполнение операций", "Верный PIN: переход к операциям");

   ATM atm2("A2", 5000, 0, bank, 1);
   atm2.enterPin(1);
   atm2.enterPin(2);
   atm2.enterPin(3);
   t.check(atm2.getStateName() == "Ожидание" && atm2.getAttempts() == 0,
         "Три неверных PIN: возврат в Ожидание");

   ATM atm3("A3", 5000, 0, bank, 1);
   atm3.enterPin(1111);
   atm3.finish();
   t.check(atm3.getStateName() == "Ожидание", "Отмена во время Аутентификации");
```

```
}

void testNoConnection(Tester& t) {
   Bank bank;
   fillBank(bank);
   // Вероятность отсутствия связи 100%
   ATM atm("A1", 5000, 1.0, bank, 1);
   string answer = atm.enterPin(1234);
   t.check(atm.getStateName() == "Ожидание" && answer.find("Нет связи") != string::npos,
         "Без связи с банком вход невозможен");
}

void testOperations(Tester& t) {
   Bank bank;
   fillBank(bank);
   ATM atm("A1", 5000, 0, bank, 1);
   atm.enterPin(1234);

   atm.withdraw(1000);
   t.check(atm.getCash() == 4000 && bank.findAccount(1234)->balance == 9000,
         "Снятие 1000 руб.: меньше и в банкомате, и на счете");
   atm.withdraw(150);
   t.check(atm.getCash() == 4000, "Сумма не кратная 100 не выдается");
   atm.withdraw(4500);
   t.check(atm.getCash() == 4000, "Нельзя снять больше, чем есть в банкомате");
   atm.loadMoney(1000);
   t.check(atm.getCash() == 4000, "Во время обслуживания клиента загрузка запрещена");
   atm.finish();
   t.check(atm.getStateName() == "Ожидание", "Завершение работы: возврат в Ожидание");
}

void testBalance(Tester& t) {
   Bank bank;
   bank.addAccount(1111, "Петров", 500);
   ATM atm("A1", 5000, 0, bank, 1);
   atm.enterPin(1111);
   atm.withdraw(1000);
   t.check(atm.getCash() == 5000 && bank.findAccount(1111)->balance == 500,
         "Нельзя снять больше, чем на счете");
}

void testBlocked(Tester& t) {
   Bank bank;
   fillBank(bank);
   ATM atm("A1", 2000, 0, bank, 1);
   atm.enterPin(1234);
   atm.withdraw(2000);
```

```
    t.check(atm.getStateName() == "Заблокирован" && atm.getCash() == 0,
        "Деньги закончились: банкомат заблокирован");

    atm.enterPin(1234);
    t.check(atm.getStateName() == "Заблокирован", "В заблокированном банкомате PIN не принимается");

    atm.loadMoney(10000);
    t.check(atm.getStateName() == "Ожидание" && atm.getCash() == 10000,
        "После загрузки денег банкомат разблокирован");
}

void runTests() {
    Tester t;
    cout << "\n===== Тестирование =====" << endl;
    testInitialState(t);
    testIdle(t);
    testAuthentication(t);
    testNoConnection(t);
    testOperations(t);
    testBalance(t);
    testBlocked(t);
    t.printResult();
}

// ====================== Демонстрация ======================

int main() {
    setlocale(LC_ALL, "Russian");

    Bank bank;
    bank.addAccount(1234, "Иванов", 10000);
    bank.addAccount(4321, "Петрова", 3000);

    // Вероятность отсутствия связи с банком - 10%
    random_device device;
    ATM atm("ATM-001", 5000, 0.1, bank, device());

    cout << "===== Демонстрация =====" << endl;
    cout << "Банкомат " << atm.getId() << ", состояние: " << atm.getStateName()
        << ", денег: " << atm.getCash() << " руб." << endl << endl;

    show(atm, "Снять 1000", atm.withdraw(1000));
    show(atm, "Ввести PIN 1111", atm.enterPin(1111));
    show(atm, "Ввести PIN 1234", atm.enterPin(1234));
    show(atm, "Снять 3000", atm.withdraw(3000));
    show(atm, "Завершить работу", atm.finish());
```

```
    show(atm, "Ввести PIN 4321", atm.enterPin(4321));
    show(atm, "Снять 2000", atm.withdraw(2000));
    show(atm, "Ввести PIN 4321", atm.enterPin(4321));
    show(atm, "Загрузить 20000", atm.loadMoney(20000));
    show(atm, "Ввести PIN 4321", atm.enterPin(4321));
    show(atm, "Снять 2000", atm.withdraw(2000));
    show(atm, "Завершить работу", atm.finish());

    runTests();
    return 0;
}
```